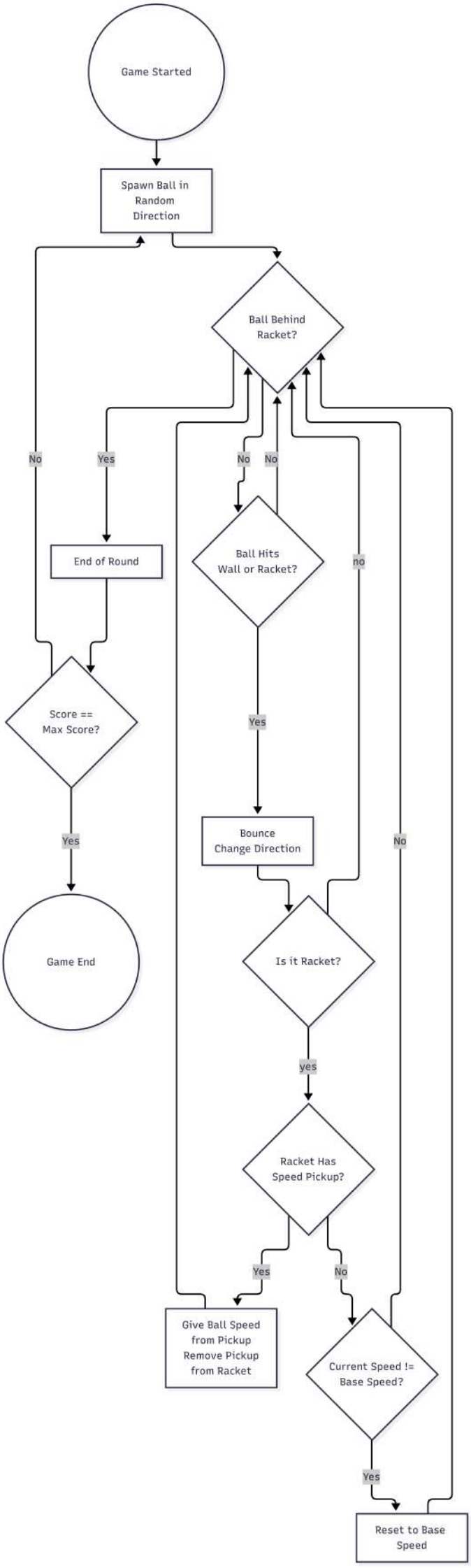
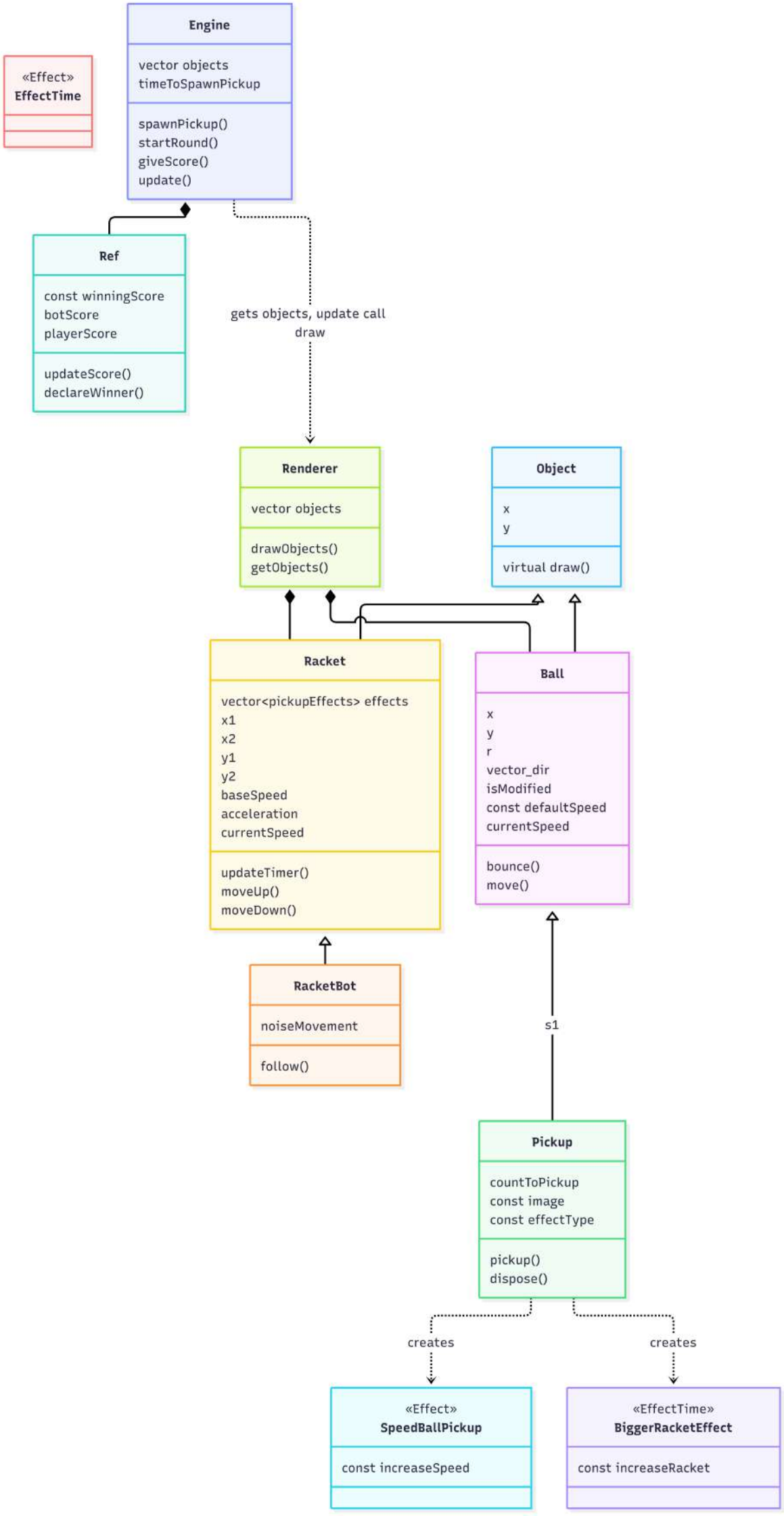
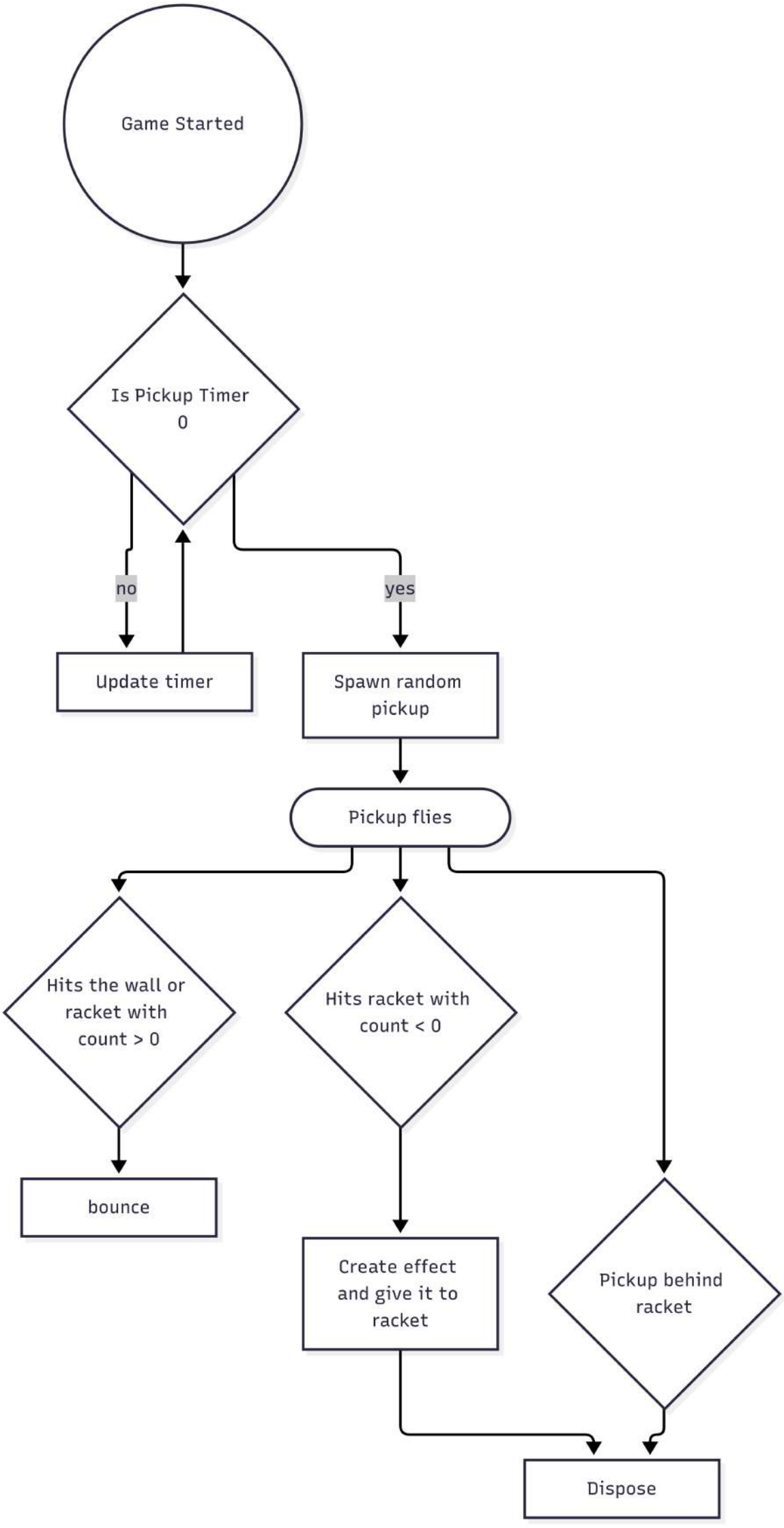
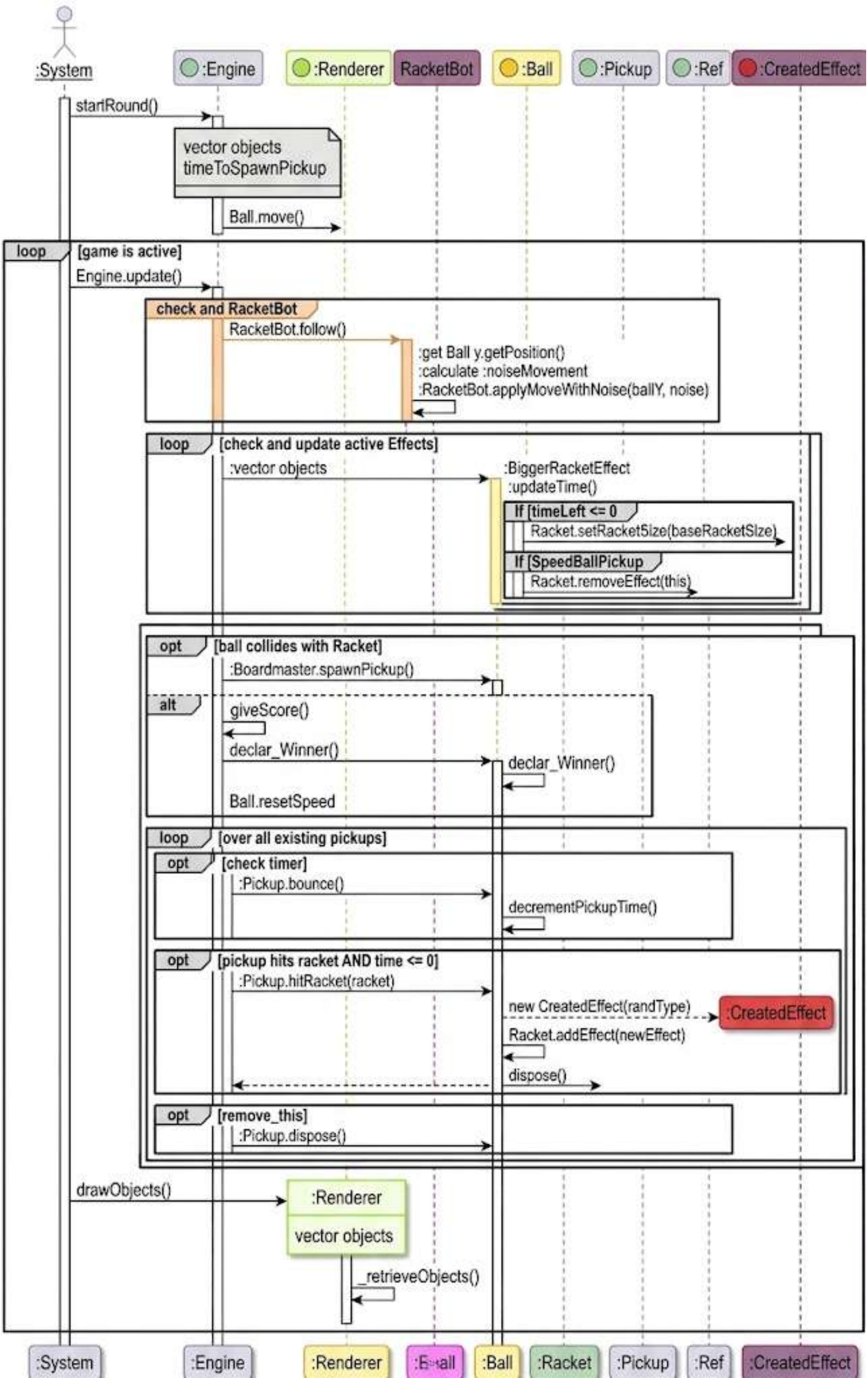
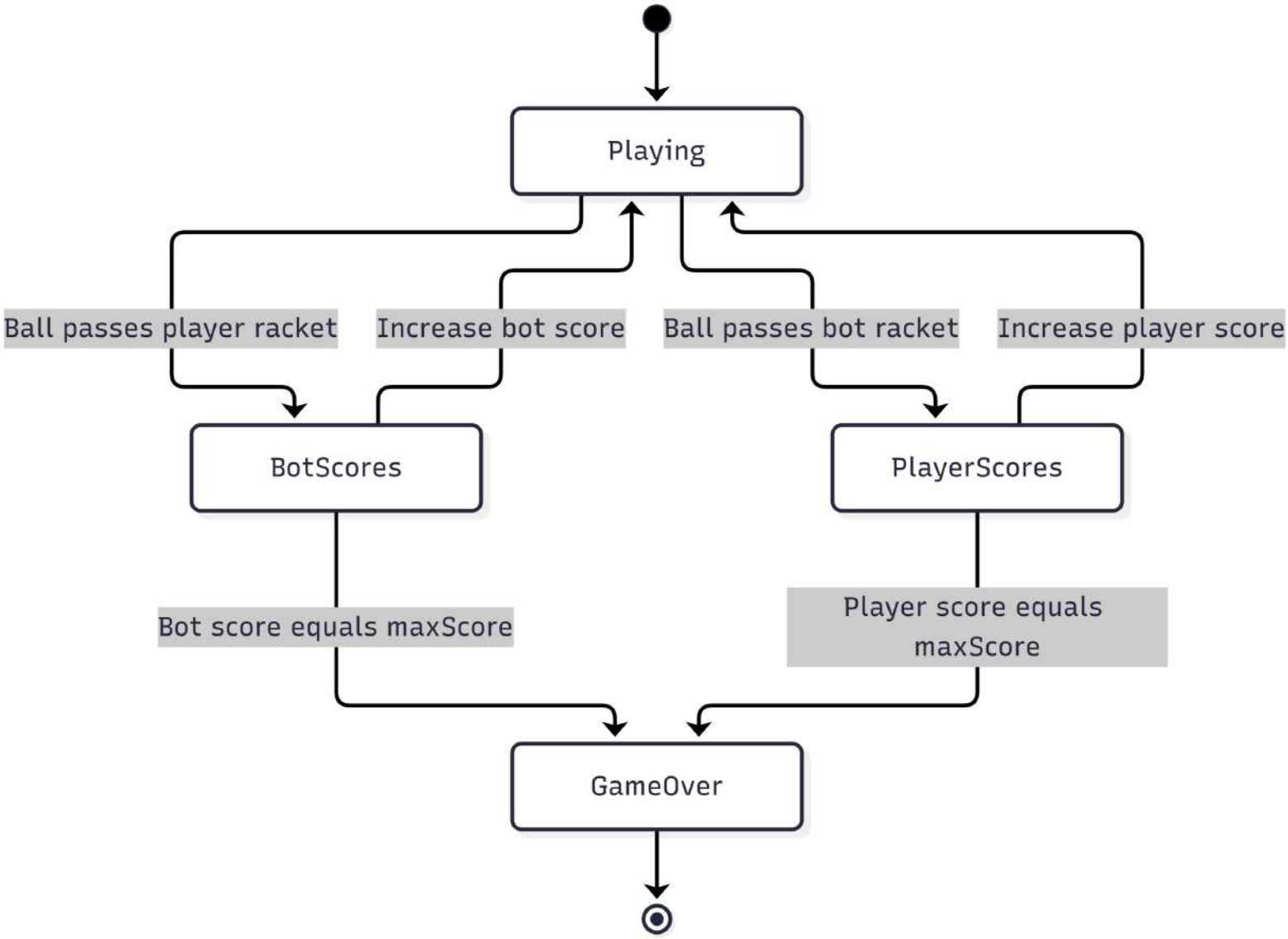
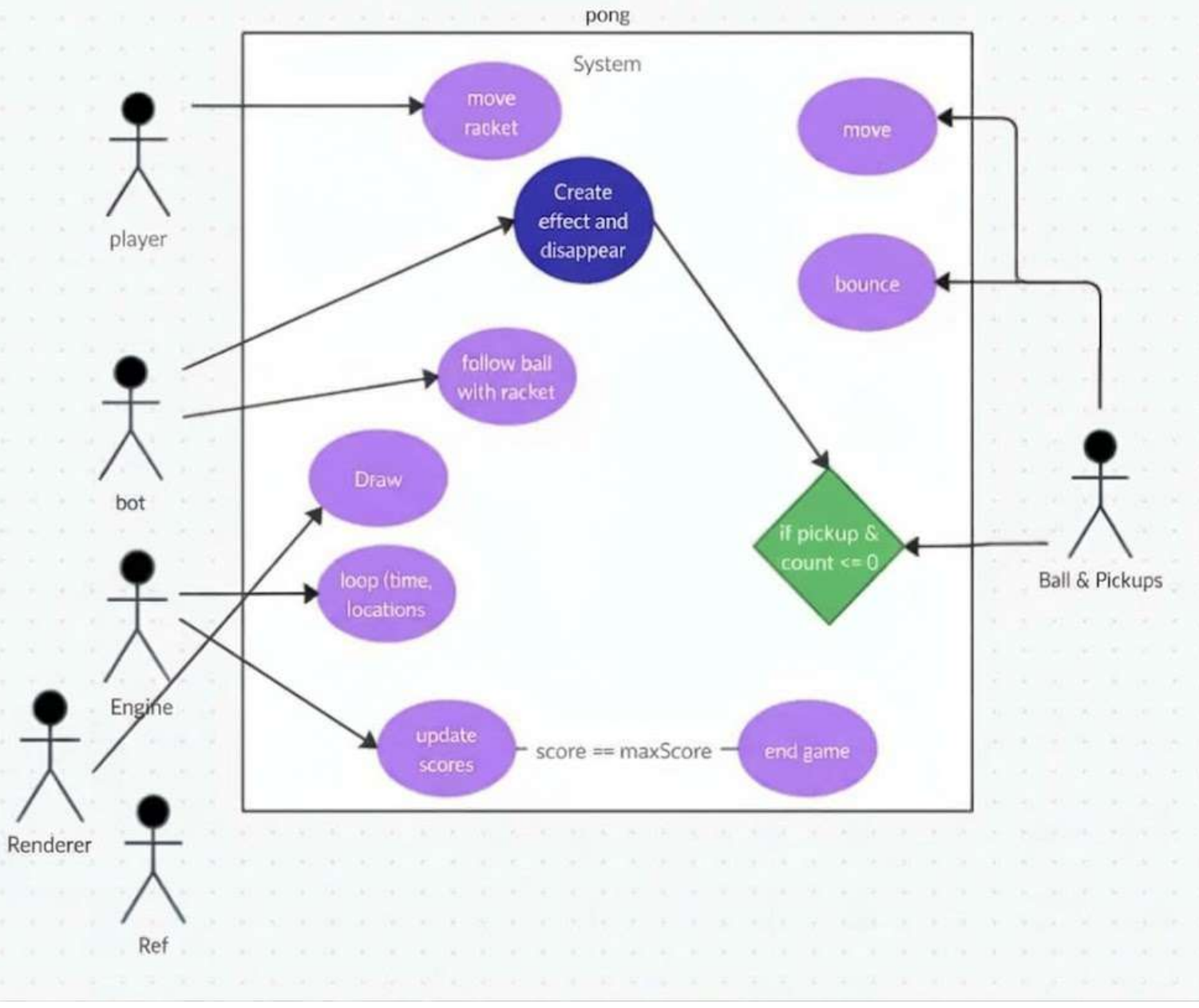

Game Started
Spawn Ball in Random Direction
Ball Behind Racket?
No
Yes
End of Round
Score == Max Score?
Yes
Game End
No
No
Ball Hits Wall or Racket?
Yes
Bounce Change Direction
Is it Racket?
yes
no
Racket Has Speed Pickup?
Yes
No
Give Ball Speed from Pickup Remove Pickup from Racket
Current Speed != Base Speed?
Yes
Reset to Base Speed
No


«Effect»
EffectTime
Engine
vector objects
timeToSpawnPickup
spawnPickup()
startRound()
giveScore()
update()
Ref
const winningScore
botScore
playerScore
updateScore()
declareWinner()
gets objects, update call
draw
Renderer
vector objects
drawObjects()
getObjects()
Object
x
y
virtual draw()
Racket
vector<pickupEffects> effects
x1
x2
y1
y2
baseSpeed
acceleration
currentSpeed
updateTimer()
moveUp()
moveDown()
Ball
x
y
r
vector_dir
isModified
const defaultSpeed
currentSpeed
bounce()
move()
RacketBot
noiseMovement
follow()
s1
Pickup
countToPickup
const image
const effectType
pickup()
dispose()
creates
creates
«Effect»
SpeedBallPickup
const increaseSpeed
«EffectTime»
BiggerRacketEffect
const increaseRacket


Game Started
Is Pickup Timer 0
no
yes
Update timer
Spawn random pickup
Pickup flies
Hits the wall or racket with count > 0
Hits racket with count < 0
bounce
Create effect and give it to racket
Pickup behind racket
Dispose


:System
:Engine
:Renderer
RacketBot
:Ball
:Pickup
:Ref
:CreatedEffect
startRound()
vector objects
timeToSpawnPickup
Ball.move()
loop
[game is active]
Engine.update()
check and RacketBot
RacketBot.follow()
:get Ball y.getPosition()
:calculate :noiseMovement
:RacketBot.applyMoveWithNoise(ballY, noise)
loop
[check and update active Effects]
:vector objects
:BiggerRacketEffect
:updateTime()
If [timeLeft <= 0
Racket.setRacketSize(baseRacketSize)
If [SpeedBallPickup
Racket.removeEffect(this)
opt
[ball collides with Racket]
:Boardmaster.spawnPickup()
alt
giveScore()
declar_Winner()
declar_Winner()
Ball.resetSpeed
loop
[over all existing pickups]
opt
[check timer]
:Pickup.bounce()
decrementPickupTime()
opt
[pickup hits racket AND time <= 0]
:Pickup.hitRacket(racket)
new CreatedEffect(randType)
:CreatedEffect
Racket.addEffect(newEffect)
dispose()
opt
[remove_this]
:Pickup.dispose()
drawObjects()
:Renderer
vector objects
_retrieveObjects()
:System
:Engine
:Renderer
:Ball
:Racket
:Pickup
:Ref
:CreatedEffect


Playing
Ball passes player racket
Increase bot score
Ball passes bot racket
Increase player score
BotScores
PlayerScores
Bot score equals maxScore
Player score equals maxScore
GameOver


pong
System
move racket
move
Create effect and disappear
bounce
follow ball with racket
Draw
if pickup & count <= 0
loop (time, locations
update scores
score == maxScore
end game
player
bot
Engine
Renderer
Ref
Ball & Pickups